



please explain entire file in details with examples for each step for better understanding. Refer attached file.

You've built a pretty sophisticated workflow here, so focusing on the high-level flow plus a concrete example for each stage is the best way to internalize it.^[1]

Assume this situation:

- App: hnw-services-2
- Branches in repo:
 - release/hnw-services-2_1.00.00
 - release/hnw-services-2_1.02.03
 - release/hnw-services-2_1.2.03
- You just merged PR #123 into release/hnw-services-2_1.02.03.

The goal: automatically create (and possibly merge) a cascade PR from release/hnw-services-2_1.02.03 → release/hnw-services-2_1.2.03, with careful mergeability handling.

1. When and how the workflow runs

```
name: Cascade Next PR

on:
  pull_request_target:
    types: [closed]
```

- Triggers when any PR is closed (merged or not).
- Uses pull_request_target so the workflow runs as the base repo and has access to secrets.CASCADE_GITHUB_TOKEN, even when the PR comes from a fork.^[2]

```
concurrency:
  group: cascade-next-${{ github.event.pull_request.number || github.run_id }}
  cancel-in-progress: true
```

- Ensures there is at most one active run per PR.
- For PR #123, group → cascade-next-123. If a new run starts for the same PR, the older one is canceled.

```
permissions:
  contents: read
  pull-requests: write
  issues: write
```

- Grants the workflow:
 - Read access to branches (listBranches).
 - Write on PRs (create, merge).
 - Write on issues (labels, comments).

```
jobs:
  cascade-next:
    if: github.event.pull_request.merged == true
    runs-on: ubuntu-latest
```

- Job runs only if the closed PR was merged (ignores closed-without-merge).
- Uses GitHub-hosted Ubuntu runner.

2. The single job step: context gathering

```
steps:
  - name: Create and process next cascade PR
    uses: actions/github-script@v7
    with:
      github-token: ${ secrets.CASCADE_GITHUB_TOKEN }
    script: |
      const owner = context.repo.owner;
      const repo = context.repo.repo;

      const baseBranch = context.payload.pull_request?.base?.ref;
      const mergedPrNumber = context.payload.pull_request?.number;

      core.info(`Event: ${context.eventName}`);
      core.info(`Base branch: ${baseBranch}`);
      core.info(`Merged PR number: ${mergedPrNumber}`);

      if (!baseBranch) {
        core.setFailed('Base branch could not be determined.');
```

In our example:

- owner = "your-org"
- repo = "your-repo"
- baseBranch = "release/hnw-services-2_1.02.03"
- mergedPrNumber = 123

If baseBranch is missing (malformed event), step fails early.

3. Branch pattern & version parsing

Regex: what counts as a release branch

```
const releaseRegex =  
  /^release\/(?<app>[A-Za-z0-9.-]+)_(?<version>\d{1,3}\.? \d{0,2}\.? \d{0,2})(?:[-_].*)
```

- Must start with `release/`.
- `app`: `[A-Za-z0-9.-]+` (letters, digits, dots, dashes; no underscore).
- `_` separator.
- `version`:
 - 1–3 digits (major), then up to two optional “XX” segments:
 - Examples:
 - 1
 - 1.2
 - 1.02
 - 1.2.03
 - 1.00.00
- Optional suffix: `-hotfix`, `_beta`, etc.

So:

- `release/hnw-services-2_1.02.03` → `app=hnw-services-2`, `version=1.02.03`
- `release/hnw-services-2_1.2.03` → `version=1.2.03`
- `release/hnw-services-2_1.00.00` → `version=1.00.00`

`parseBranch`: turn branch name into numeric `versionParts`

```
function parseBranch(branch) {  
  const match = branch.match(releaseRegex);  
  if (!match) return null;  
  
  const { app, version } = match.groups;  
  
  const parts = version.split('.'); // ["1"], ["1","2"], ["1","2","03"]  
  
  function toMinorOrPatchValue(str) {  
    if (str === undefined || str.length === 0) {  
      return 0; // missing -> 0  
    }  
    if (str.length === 2) {  
      // 2 digits: plain number, "02" -> 2, "99" -> 99  
    }  
  }  
}
```

```

    return parseInt(str, 10);
  }
  // 1 digit: tens, "2" -> 20, "3" -> 30, etc.
  const digit = parseInt(str, 10);
  return digit * 10;
}

const major = parseInt(parts[0], 10);
const minor = toMinorOrPatchValue(parts[1]);
const patch = toMinorOrPatchValue(parts[2]);

const versionParts = [major, minor, patch];

return {
  branch,
  app,
  version,
  versionParts
};
}

```

Using our three branches:

1.release/hnw-services-2_1.00.00:

- `version = "1.00.00"`
- `parts = ["1", "00", "00"]`
- `major = 1`
- `minor = toMinorOrPatchValue("00"):`
 - `len 2 → parseInt("00",10) = 0`
- `patch = toMinorOrPatchValue("00") → 0`
`→ versionParts = [1, 0, 0]`

2.release/hnw-services-2_1.02.03:

- `version = "1.02.03" → ["1", "02", "03"]`
- `major = 1`
- `minor = "02" → len 2 → parseInt("02",10) = 2`
- `patch = "03" → 3`
`→ versionParts = [1, 2, 3]`

3.release/hnw-services-2_1.2.03:

- `version = "1.2.03" → ["1", "2", "03"]`
- `major = 1`
- `minor = "2" → len 1 → parseInt("2",10)=2 → 2 * 10 = 20`
- `patch = "03" → 3`
`→ versionParts = [1, 20, 3]`

So your “semantic” comparison is:

- 1.00.00 → [1,0,0]
- 1.02.03 → [1,2,3]
- 1.2.03 → [1,20,3] (treated like 1.20.03).

4. compareVersionOnly: numeric sort of these versions

```
function compareVersionOnly(a, b) {
  const len = Math.max(a.versionParts.length, b.versionParts.length);
  for (let i = 0; i < len; i++) {
    const av = a.versionParts[i] ?? 0;
    const bv = b.versionParts[i] ?? 0;
    if (av < bv) return -1;
    if (av > bv) return 1;
  }
  return 0;
}
```

Compare:

- [1,0,0] vs [1,2,3]:
 - index 0:1 vs 1 → equal
 - index 1:0 vs 2 → 0 < 2 → first is smaller.
- [1,2,3] vs [1,20,3]:
 - index 0:1 vs 1 → equal
 - index 1:2 vs 20 → 2 < 20 → [1,2,3] smaller.

So sorting these three versions gives:

```
1.1.00.00
2.1.02.03
3.1.2.03
```

5. Helpers: sleep, getPullRequest, pollMergeability

```
async function sleep(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}
```

- Simple async delay.

```
async function getPullRequest(prNumber) {
  const response = await github.rest.pulls.get({ owner, repo, pull_number: prNumber });
  return response.data;
}
```

- Wrapper for GET /repos/{owner}/{repo}/pulls/{number} to get PR details.^[3]

```
async function pollMergeability(prNumber, maxAttempts = 10, delayMs = 2000) {
  let pr = null;

  for (let attempt = 1; attempt <= maxAttempts; attempt++) {
    pr = await getPullRequest(prNumber);

    core.info(
      `Poll ${attempt}/${maxAttempts} for PR #${prNumber}: mergeable=${pr.mergeabl
    );

    if (pr.mergeable !== null) {
      return pr;
    }

    if (attempt < maxAttempts) {
      await sleep(delayMs);
    }
  }

  return pr;
}
```

- Polls the new cascade PR until GitHub has computed `mergeable` and `mergeable_state`, or until the polling window expires.

6. current & eligible release branches

```
const current = parseBranch(baseBranch);
if (!current) {
  core.info(`Base branch '${baseBranch}' does not match the release cascade pattern.
  return;
}
```

- For baseBranch `release/hnw-services-2_1.02.03`, current is the parsed object with `versionParts = [1,2,3]`.

```
const branches = await github.paginate(github.rest.repos.listBranches, {
  owner,
  repo,
  per_page: 100
});
```

- Fetches all branches with pagination.

```
const eligibleBranches = branches
  .map(branch => parseBranch(branch.name))
  .filter(Boolean)
```

```
.filter(branch => branch.app === current.app)
.sort(compareVersionOnly);
```

- `parseBranch` returns either a parsed object or `null`.
- `filter(Boolean)` removes non-release branches.
- `filter(branch.app === current.app)` keeps only the same app (hnw-services-2).
- `sort(compareVersionOnly)` sorts by normalized version.

For our example, `eligibleBranches` ends up as an array of 3 objects ordered as:

- 1.release/hnw-services-2_1.00.00 → [1,0,0]
- 2.release/hnw-services-2_1.02.03 → [1,2,3]
- 3.release/hnw-services-2_1.2.03 → [1,20,3]

```
if (eligibleBranches.length === 0) {
  core.info(`No eligible release branches found for app '${current.app}'. Exiting.`
  return;
}

core.info(
  `Eligible ordered branches for app '${current.app}': ${eligibleBranches
    .map(branch => branch.branch)
    .join(' -> ')}`
);
```

- Logs:
 - Eligible ordered branches for app 'hnw-services-2': release/hnw-services-2_1.00.00 -> release/hnw-services-2_1.02.03 -> release/hnw-services-2_1.2.03

7. Picking nextBranch (or develop)

```
const currentIndex = eligibleBranches.findIndex(branch => branch.branch === current.branch);
if (currentIndex === -1) {
  core.setFailed(`Current branch '${current.branch}' was not found in the eligible branches`);
  return;
}
```

- For baseBranch `release/hnw-services-2_1.02.03`, `currentIndex = 1`.

```
let nextBranch;
let isFinalMergeToDevelop = false;

if (currentIndex === eligibleBranches.length - 1) {
  nextBranch = 'develop';
  isFinalMergeToDevelop = true;
  core.info(`Branch '${current.branch}' is the last eligible release branch. Final P`
} else {
  nextBranch = eligibleBranches[currentIndex + 1].branch;
```

```
core.info(`Next cascade target branch: ${nextBranch}`);
}
```

- Since `currentIndex = 1` and `eligibleBranches.length - 1 = 2`, not last.
- So `nextBranch = eligibleBranches[^2].branch = "release/hnw-services-2_1.2.03"`.
- `isFinalMergeToDevelop = false`.

If you merged into the last release branch, it would instead set `nextBranch = "develop"` and `isFinalMergeToDevelop = true`.

8. Avoid duplicate cascade PRs

```
const openPullRequests = await github.paginate(github.rest.pulls.list, {
  owner,
  repo,
  state: 'open',
  base: nextBranch,
  per_page: 100
});

const existingPr = openPullRequests.find(pr => pr.head.ref === current.branch);
if (existingPr) {
  core.info(`An open cascade PR already exists: #${existingPr.number} (${current.branch})`);
  return;
}
```

- Finds any open PR whose:
 - base is `nextBranch`
 - head branch is `current.branch`
- If found, logs and exits (prevents duplicate cascade PRs).

9. Creating the cascade PR

```
const title = isFinalMergeToDevelop
  ? `[Cascade Final] ${current.branch} -> ${nextBranch}`
  : `[Cascade] ${current.branch} -> ${nextBranch}`;
```

- If final to develop, title starts with `[Cascade Final]`.
- Otherwise just `[Cascade]`.

```
const body = [
  `This PR was created automatically after PR #${mergedPrNumber} was merged into \`${current.branch}\`,`,
  `### Cascade behavior`,
  isFinalMergeToDevelop
    ? `- This is the final cascade PR from the last eligible release branch into \`${current.branch}\`.`
    : `- Only the next downstream cascade PR is created.`
].join('\n');
```



```

    '- If this PR has conflicts, the cascade stops here.',
    '- If this PR is immediately mergeable, it is merged automatically.',
    '- If required branch conditions block merge, the PR remains open for review.',
    isFinalMergeToDevelop
    ? '- Once this PR is merged, the release cascade is complete.'
    : '- Once this PR is merged, the next downstream cascade PR can be created autom
    '',
    '### Details',
    '- App: `${current.app}`,
    '- Current version: `${current.version}`,
    '- Source branch: `${current.branch}`,
    '- Target branch: `${nextBranch}`
  ].join('\n');

```

- PR body explains:
 - Which PR triggered this.
 - Cascade behavior rules.
 - App, version, source & target branch.

```

let createdPr;
try {
  createdPr = await github.rest.pulls.create({
    owner,
    repo,
    head: current.branch,
    base: nextBranch,
    title,
    body,
    maintainer_can_modify: true
  });
} catch (error) {
  if (error.status === 422) {
    core.warning(
      `Cascade PR was not created. This may mean there is no diff or an equivalent PR
    );
    return;
  }
  throw error;
}

const prNumber = createdPr.data.number;
core.info(`Created cascade PR #${prNumber}`);

await addLabel(prNumber, isFinalMergeToDevelop ? 'cascade-final-pr' : 'cascade-pr')

```

- Creates `current.branch` -> `nextBranch` PR.
- On 422 (no diff or existing equivalent PR), logs and exits.
- Labels it `cascade-final-pr` OR `cascade-pr`.

10. Mergeability logic for the new cascade PR

```
let prDetails = await pollMergeability(prNumber, 10, 2000);

core.info(
  `Final mergeability for PR #${prNumber}: mergeable=${prDetails?.mergeable}, mergeable_state=${prDetails?.mergeable_state}`
);
```

- Polls and logs final mergeable and mergeable_state.

Case A: still unknown

```
if (prDetails?.mergeable === null || prDetails?.mergeable_state === 'unknown') {
  await addLabel(prNumber, 'cascade-pending');
  await addComment(
    prNumber,
    [
      `The workflow could not determine mergeability for this cascade PR within the`,
      ``,
      `The PR has been left open for manual review or a later retry.`,
      `Observed state: mergeable=\`${prDetails?.mergeable}\`, mergeable_state=\`${prDetails?.mergeable_state}\``
    ].join('\n')
  );
  core.info(`PR #${prNumber} remained unresolved after polling. Leaving it open.`);
  return;
}
```

- Could not determine mergeability → label cascade-pending, comment, leave open.

Case B: conflicts / dirty

```
if (prDetails?.mergeable === false || prDetails?.mergeable_state === 'dirty') {
  await addLabel(prNumber, 'cascade-conflict');
  await addComment(
    prNumber,
    [
      `This cascade PR. 目前有冲突或无法合并.`,
      ``,
      `isFinalMergeToDevelop`,
      `? `The final merge into \`${nextBranch}\` is paused until this PR is resolved`,
      `: `The cascade stops here until this PR is resolved and merged.`
    ].join('\n')
  );
  core.info(`PR #${prNumber} is not mergeable. Cascade stops here.`);
  return;
}
```

- Conflicts → cascade-conflict, cascade stops until resolved and merged.

Case C: clean

```
if (prDetails?.mergeable === true && prDetails?.mergeable_state === 'clean') {
  await mergePullRequest(prNumber);
  core.info(`PR #${prNumber} was merged immediately because it is in clean state.`)
  return;
}
```

- Clean → auto-merge.

Case D: blocked

```
if (prDetails?.mergeable === true && prDetails?.mergeable_state === 'blocked') {
  await addLabel(prNumber, 'cascade-blocked');
  await addComment(
    prNumber,
    [
      `This cascade PR is currently blocked by required merge conditions or repository rules`,
      ``,
      `The workflow will not force-merge this PR.`,
      `Observed state: mergeable=${prDetails?.mergeable}`, mergeable_state=${prDetails?.mergeable_state}
    ].join('\n')
  );
  core.info(`PR #${prNumber} is blocked. Leaving it open for required conditions to be met.`)
  return;
}
```

- Blocked by rules (e.g. status checks) → cascade-blocked, leave open.

Case E: unstable (flaky checks etc.)

```
if (prDetails?.mergeable === true && prDetails?.mergeable_state === 'unstable') {
  const unstableMaxAttempts = 12;
  const unstableDelayMs = 10000;

  core.info(`PR #${prNumber} is in unstable state. Retrying for up to ${unstableMaxAttempts} attempts.`)

  for (let attempt = 1; attempt <= unstableMaxAttempts; attempt++) {
    await sleep(unstableDelayMs);
    prDetails = await getPullRequest(prNumber);

    core.info(
      `Unstable retry ${attempt}/${unstableMaxAttempts} for PR #${prNumber}: mergeable=${prDetails?.mergeable}, mergeable_state=${prDetails?.mergeable_state}`
    );

    if (prDetails?.mergeable === true && prDetails?.mergeable_state === 'clean') {
      await mergePullRequest(prNumber);
      core.info(`PR #${prNumber} was merged after unstable state resolved to clean.`)
      return;
    }
  }

  if (prDetails?.mergeable === false || prDetails?.mergeable_state === 'dirty') {
    core.info(`PR #${prNumber} is in dirty state. Leaving it open for required conditions to be met.`)
    return;
  }
}
```

```

    await addLabel(prNumber, 'cascade-conflict');
    await addComment(
      prNumber,
      [
        `This cascade PR became non-mergeable while waiting for unstable state to r
        `,
        isFinalMergeToDevelop
          ? `The final merge into `${nextBranch}` is paused until this PR is reso
          : `The cascade stops here until this PR is resolved and merged.`,
        `Observed state: mergeable=`${prDetails?.mergeable}`, mergeable_state=
      ].join('\n')
    );
    core.info(`PR #${prNumber} became non-mergeable during unstable retry window.
    return;
  }

  if (prDetails?.mergeable === true && prDetails?.mergeable_state === 'blocked') {
    await addLabel(prNumber, 'cascade-blocked');
    await addComment(
      prNumber,
      [
        `This cascade PR became blocked by required merge conditions or repositor
        `,
        `The workflow will not force-merge this PR.`,
        `Observed state: mergeable=`${prDetails?.mergeable}`, mergeable_state=
      ].join('\n')
    );
    core.info(`PR #${prNumber} became blocked during unstable retry window.`);
    return;
  }
}

await addLabel(prNumber, 'cascade-unstable');
await addComment(
  prNumber,
  [
    `This cascade PR remained in an unstable state and was not merged automatically
    `,
    `The workflow retried for a longer window, but the PR did not become clean.`,
    `Observed state: mergeable=`${prDetails?.mergeable}`, mergeable_state=`${pr
  ].join('\n')
);
core.info(`PR #${prNumber} remained unstable after extended retry window. Leaving
return;
}

```

- For unstable:
 - Re-polls for a longer period (12 tries, 10s each).
 - If it becomes clean → auto-merge.
 - If it becomes dirty → conflict.
 - If it becomes blocked → blocked.
 - If still unstable → label `cascade-unstable`, comment, leave open.

Case F: anything else (fallback)

```
await addLabel(prNumber, 'cascade-manual-review');
await addComment(
  prNumber,
  [
    `This cascade PR reached an unhandled mergeability state and requires manual review`,
    `Observed state: mergeable=\`${prDetails?.mergeable}\`, mergeable_state=\`${prDetails?.mergeable_state}\`, mergeable_state_reason=\`${prDetails?.mergeable_state_reason}\``,
  ].join('\n')
);
core.warning(`PR #${prNumber} reached an unhandled state. Leaving it open.`);
```

- Catches any weird/unexpected mergeability combination.

All together:

- You merge PR → workflow runs.
- It checks base release branch name and parses version → fits your custom version semantics.
- It determines the next branch (or develop) in the sorted version timeline for that app.
- It creates exactly one cascade PR for current → next.
- It then carefully inspects mergeability and either:
 - auto-merges,
 - labels as pending/conflict/blocked/unstable,
 - or routes to manual review.

To check you're fully comfortable: if your release branches were:

- release/hnw-services-2_1.00.00
- release/hnw-services-2_1.09.00
- release/hnw-services-2_1.9.00

what versionParts would each get, and in what order would they appear in eligibleBranches after sorting?

✱

1. cascade-next-pr.yml
2. <https://stackoverflow.com/questions/74957218/what-is-the-difference-between-pull-request-and-pull-request-target-event-in-git>
3. <https://docs.github.com/en/rest/pulls/pulls>